



程序设计思维与实践

Thinking and Practice in Programming

树形数据结构及其应用 | 内容负责：李子豪



1

问题引入

Introduce with questions

引入

- 给一个数组 $a[1]..a[n]$ ，有 m 次修改，每次修改给出一个区间 $[1, r]$ 和一个数 k ，要把 $a[1..r]$ 都加上 k ， m 次修改后请你输出这 n 个数， $n, m \leq 10^6$
- 差分， $O(N + M)$ 。
- 给一个数组 $a[1]..a[n]$ ，有 m 次查询，每次查询给出一个区间 $[1, r]$ ，询问区间内元素的和 $(a[1] + a[1+1] + \dots + a[r])$ ， $n, m \leq 10^6$
- 前缀和， $O(N + M)$ 。

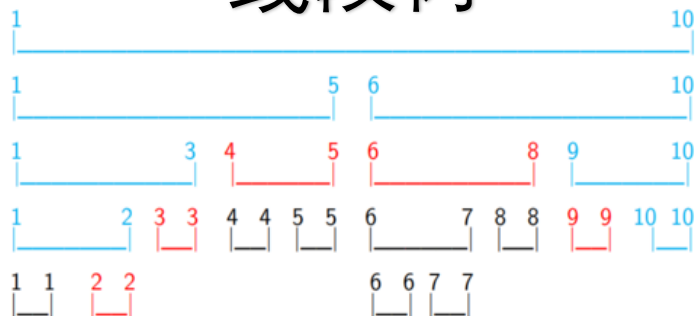
引入

- **单点修改, 区间查询** — 给一个数组 $a[1]..a[n]$, 有 m 次操作, 每次操作可能是:
 - $1\ x\ k$: 表示将 $a[x]$ 加上 k
 - $2\ l\ r$: 询问 $a[l]$ 到 $a[r]$ 的和
- **区间修改, 单点查询** — 给一个数组 $a[1]..a[n]$, 有 m 次操作, 每次操作可能是:
 - $1\ l\ r\ k$: 表示将 $a[l]$ 到 $a[r]$ 加上 k
 - $2\ x$: 询问 $a[x]$ 的值
- **区间修改, 区间查询** — 给一个数组 $a[1]..a[n]$, 有 m 次操作, 每次操作可能是:
 - $1\ l\ r\ k$: 表示将 $a[l]$ 到 $a[r]$ 加上 k
 - $2\ l\ r$: 询问 $a[l]$ 到 $a[r]$ 的和
- 以上问题差分/前缀和均退化到 $O(NM)$ 。

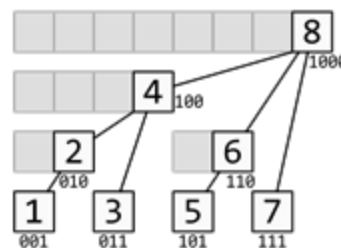
引入

- 差分、前缀和只能支持快速的修改/查询，而不能支持二者穿插操作
- 我们需要一些能够支持修改/查询穿插操作的算法

线段树



树状数组



- 线段树能做的问题：区间加、区间乘、区间求和、区间最值、区间覆盖、01串区间取反、区间最大子段和..... 还有权值树、动态开点、可持久化等成吨的变种和拓展功能，以及可以和其他数据结构相互嵌套。
- 树状数组能做的问题线段树都能做，反之则不然。但是树状数组实现简单（核心代码仅几行），时空常数小，因此仍有学习的必要。



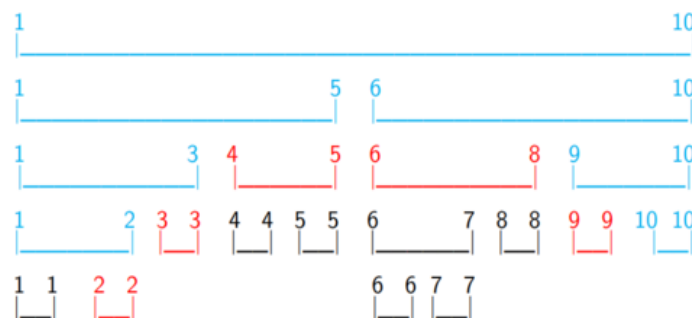
2

线段树基础

Foundation of Segment Tree

线段树的结构

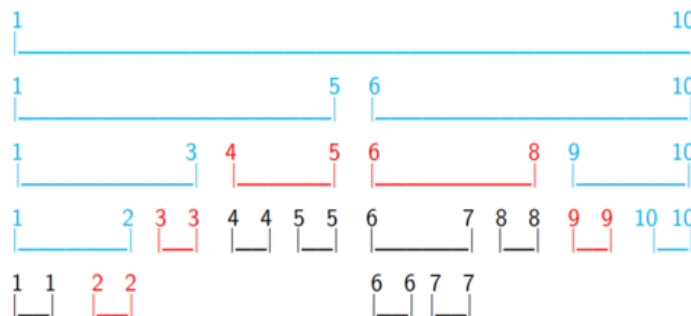
- **树**：n个点n-1条边构成的无向无环连通图称之为“树”
- **有根树**：在树中钦定一个点为根得到的树为“有根树”，并且从根出发向下，每个点的下层顶点为这个点的儿子节点，没有儿子的节点称为叶子节点
- **二叉树**：若一棵有根树中每个节点的儿子数目不超过2，那么这棵树称为二叉树
- 二叉树的**二进制编号**：对于一颗二叉树，我们将根节点编号为1，之后对于节点i，若其有左儿子，则将其左儿子编号为 $2*i$ ，若其有右儿子，则将其右儿子编号为 $2*i+1$ ，如此得到的编号我们称之为二进制编号。
- **(静态) 线段树是一颗采用二进制标号的二叉树**



线段树的结构

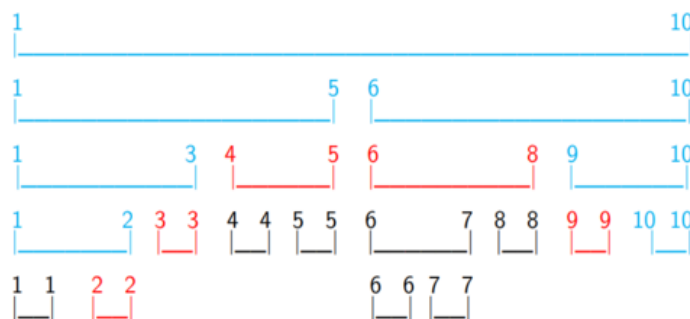
• 线段树的构建：

- 建立一棵二叉树，每个点对应序列中的一个区间，其中根对应总区间 $[1, n]$
- 对于一个点，若其对应的区间为 $[1, r]$ ：
 - 若 $l==r$ ，则这个点为叶子节点
 - 否则这个点有两个儿子，左儿子对应区间 $[1, mid]$ ，右儿子对应区间 $[mid + 1, r]$ ，其中 $mid = (1 + r) / 2$ （蕴含着二分、分治的思想）
- 每个节点处记录这个节点对应区间的**信息**（如区间和、区间最值等）
- 使用**二进制编号**可以得到一个点的左右儿子的编号



线段树的结构特点

- 每个节点要么没有儿子，要么有两个儿子
- 叶子节点的数目为 n （分别对应 $[1, 1]$, $[2, 2]$, ..., $[n, n]$ ）
- 节点总数为 $O(N)$ ，确切地说为 $2n-1$ （为什么？考虑把叶子两两合并来得到树）
- 线段树的树高为 $O(\log N)$ （为什么？考虑折半的建树过程）
- $[1, n]$ 的任意一个子区间，一定可以用 $O(\log N)$ 个线段树上的节点对应的区间恰好覆盖（为什么？考虑使用极长的线段去覆盖）
- 以上特点，使得线段树（在额外操作 $O(1)$ 的情况下）具有 $O(N)$ 的建树复杂度，和 $O(\log N)$ 的单次单点/区间操作复杂度





3

线段树实现

Implementation of Segment Tree

存储和建树

- 使用二进制编号，我们可以将线段树存储在一个数组里
- 对一个长为 n 的区间建立线段树，二进制标号最大值接近 $4n$ ，故需要开四倍的空间
- 建树用递归的方式进行，实际上就是遍历了二叉树，时间复杂度 $O(N)$
- 代码（以维护区间和为例）：

```
struct SegmentTree { // 封装整棵线段树
    struct Node { // 封装单个节点
        int l, r;
        long long sum;
    } node[N << 2]; // 二进制编号做数组下标存储节点
    void build(int i, int l, int r) {
        node[i].l = l, node[i].r = r; // 记录区间端点
        if (l == r) { // 叶子结点直接取原数组的值
            node[i].sum = a[i];
            return;
        }
        int mid = (l + r) / 2; // 区间中点
        build(2 * i, l, mid); // 递归建左子树
        build(2 * i + 1, mid + 1, r); // 递归建右子树
        node[i].sum = node[2 * i].sum + node[2 * i + 1].sum;
    }
};
```

存储和建树

- 我更习惯宏定义代替变量、只封装树、不封装节点，且不存储区间端点（用递归时附加传参代替）的写法，写起来简短得多，后续将按这种风格进行
- 存储、建树代码（以维护区间和为例）：

```
struct SegmentTree {  
    #define ls (i << 1)  
    #define rs (ls | 1)  
    #define mid (l + r >> 1)  
    long long sum[N << 2];  
    void build(int i, int l, int r) {  
        if (l == r) cin >> sum[i]; // 直接输入，省掉数组  
        else {  
            build(ls, l, mid), build(rs, mid + 1, r);  
            sum[i] = sum[ls] + sum[rs];  
        }  
    }  
}
```


单点加法 区间求和

- 给一个数组 $a[1]..a[n]$ ，有 m 次操作，每次操作可能是：
 - $1 \ x \ k$: 表示将 $a[x]$ 加上 k
 - $2 \ l \ r$: 询问 $a[1]$ 到 $a[r]$ 的和
- 单点加法**: 给从根到 $[x, x]$ 的叶子这一条链上，共 $O(\log N)$ 个节点，加上 k 即可
- 区间求和**: 用极长的线段覆盖 $[1, r]$ ，求这 $O(\log N)$ 条线段的和即可

```
void add(int i, int l, int r, int p, int k) {
    sum[i] += k;
    if (l == r) return;
    if (p ≤ mid) add(ls, l, mid, p, k);
    else add(rs, mid + 1, r, p, k);
}

long long query(int i, int l, int r, int L, int R) {
    if (L ≤ l && r ≤ R) return sum[i];
    if (l > R || r < L) return 0;
    return query(ls, l, mid, L, R) + query(rs, mid + 1, r, L, R);
}
```

- i 为当前节点的二进制编号， $[l, r]$ 为当前节点对应的区间， p, k 表示要将 $a[p]$ 加上 k ， $[L, R]$ 为要查询和的区间
- 先递归、再讨论的写法，常数略大，但代码简洁

单点加法 区间求和

- 完整代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 5e5 + 10;
4
5  struct SegmentTree {
6      #define ls (i << 1)
7      #define rs (ls | 1)
8      #define mid (l + r >> 1)
9      long long sum[N << 2];
10     void build(int i, int l, int r) {
11         if (l == r) cin >> sum[i]; // 直接输入, 省掉数组
12         else {
13             build(ls, l, mid), build(rs, mid + 1, r);
14             sum[i] = sum[ls] + sum[rs];
15         }
16     }
17     void add(int i, int l, int r, int p, int k) {
18         sum[i] += k;
19         if (l == r) return;
20         if (p <= mid) add(ls, l, mid, p, k);
21         else add(rs, mid + 1, r, p, k);
22     }

```

```

23     long long query(int i, int l, int r, int L, int R) {
24         if (L <= l && r <= R) return sum[i];
25         if (l > R || r < L) return 0;
26         return query(ls, l, mid, L, R) + query(rs, mid + 1, r, L, R);
27     }
28 } seg;
29
30 int main() {
31     int n, m;
32     cin >> n >> m;
33     seg.build(1, 1, n);
34     while (m--) {
35         int op, x, y;
36         cin >> op >> x >> y;
37         if (op == 1) seg.add(1, 1, n, x, y);
38         else cout << seg.query(1, 1, n, x, y) << '\n';
39     }
40
41     return 0;
42 }

```

- 声明 SegmentTree 类型的变量 seg, 通过调用成员函数的方式操作线段树
- 单次修改/查询时间复杂度均为 $O(\log N)$



带标记的线段树

Segment Tree with Tags

区间加法 区间求和

- 给一个数组 $a[1]..a[n]$ ，有 m 次操作，每次操作可能是：
 - 1 1 r k ：表示将 $a[1]$ 到 $a[r]$ 加上 k
 - 2 1 r ：询问 $a[1]$ 到 $a[r]$ 的和（单点查询？可以视作区间查询的特例）
- 区间求和如法炮制，时间复杂度 $O(\log N)$
- 区间加法？
 - 暴力递归到区间里的所有叶子？
 - 单次时间复杂度： $O(N)$ ，并且比起最普通的暴力还有着不小的常数

区间加法 区间求和

- **懒标记 (lazy tag) :**
 - 对于一次区间修改，我们不直接将所有的操作执行到底，而是对修改区间分解出的极长线段，先维护该线段的信息，再打上**一个“懒标记”**
 - 一个节点如果被打上懒标记，就表示有一些修改操作只进行到了这一层，没有彻底下放到叶子；反过来说，一个节点的祖先节点中如果有懒标记，那么此时这个节点的信息并不是完整操作后的正确信息
 - 在后续的操作中，如果需要一个节点递归下去，我们就把它的懒标记**下推**给它的儿子节点，并**清空**当前节点的标记，来保证下方节点信息的正确性
 - 还有一种策略是，不进行标记的下推，而是对一路上遇到的标记进行**积累**，从而得到正确的查询结果，这种策略被称作**标记永久化**
 - 有了懒标记策略，一次区间修改就可以通过对极长线段打标记来完成，保证了单次操作的时间复杂度为 $O(\log N)$

区间加法 区间求和

- 代码（区间加法，区间求和，标记下推）：

```
void pushdown(int i, int l, int r) {
    if (!tag[i]) return; // 无tag, 直接返回
    sum[ls] += tag[i] * (mid - l + 1);
    sum[rs] += tag[i] * (r - mid); // 维护左右儿子信息
    tag[ls] += tag[i], tag[rs] += tag[i]; // 维护左右儿子tag
    tag[i] = 0; // 清空当前节点tag
}

void add(int i, int l, int r, int L, int R, long long k) {
    if (l > R || r < L) return;
    if (L ≤ l && r ≤ R) // 极长线段, 维护信息, 打tag
        sum[i] += k * (r - l + 1), tag[i] += k;
    else {
        pushdown(i, l, r); // 先下传tag
        add(ls, l, mid, L, R, k); // 递归修改左子树
        add(rs, mid + 1, r, L, R, k); // 递归修改右子树
        sum[i] = sum[ls] + sum[rs]; // 合并得到当前节点信息
    }
}

long long query(int i, int l, int r, int L, int R) {
    if (L ≤ l && r ≤ R) return sum[i];
    if (l > R || r < L) return 0;
    pushdown(i, l, r); // 先下传tag
    return query(ls, l, mid, L, R) + query(rs, mid + 1, r, L, R);
}
```

- 时间复杂度 $O(\log N)$
- 递归前先下推标记，保证进入下方节点时其信息是对的
- 回溯后合并左右儿子信息更新自己，保证自己的信息是对的
- 合并操作复杂时也可以单独封装为 update 函数
- tag 数组也要开 4 倍空间，且在建树时清空

区间乘/加 区间求和

- 对于其他问题，最关键的点就是考虑如何正确表示（含义、默认值）并下推标记
- 已知一个数列，你需要进行下面三种操作：
 - 操作1：格式：1 x y k 含义：将区间[x,y]内每个数乘上k
 - 操作2：格式：2 x y k 含义：将区间[x,y]内每个数加上k
 - 操作3：格式：3 x y 含义：输出区间[x,y]内每个数的和对p取模所得的结果
- 维护两个标记add和mul，分别表示加法和乘法tag，并**规定先乘后加**。也就是说一个数 a 的真实值是 $mul * a + add$ 。容易想到add默认值为0，mul默认值为1
- 区间和容易维护，tag如何维护？
- 区间乘：**

$$(mul * a + add) * k = mul * k * a + add * k \Rightarrow mul' = mul * k, add' = add * k$$
- 区间加：**

$$(mul * a + add) + k = mul * a + add + k \Rightarrow mul' = mul, add' = add + k$$
- 标记下推（先乘后加）：**

$$mul[son] = mul[son] * mul[fa], add[son] = add[son] * mul[fa] + add[fa]$$

区间乘/加 区间求和

- 代码 (query和前面没有任何区别)

```
void pushdown(int i, int l, int r) {
    mul[ls] = mul[ls] * mul[i] % p;
    mul[rs] = mul[rs] * mul[i] % p;
    add[ls] = (add[ls] * mul[i] + add[i]) % p;
    add[rs] = (add[rs] * mul[i] + add[i]) % p;
    sum[ls] = (sum[ls] * mul[i] + (mid - l + 1) * add[i]) % p;
    sum[rs] = (sum[rs] * mul[i] + (r - mid) * add[i]) % p;
    add[i] = 0, mul[i] = 1;
}
```

```
void Mul(int i, int l, int r, int L, int R, long long k) {
    if (l > R || r < L) return;
    else if (L ≤ l && r ≤ R) {
        sum[i] = sum[i] * k % p;
        mul[i] = mul[i] * k % p;
        add[i] = add[i] * k % p;
    }
    else {
        pushdown(i, l, r);
        Mul(ls, l, mid, L, R, k);
        Mul(rs, mid + 1, r, L, R, k);
        sum[i] = (sum[ls] + sum[rs]) % p;
    }
}
```

```
void Add(int i, int l, int r, int L, int R, long long k) {
    if (l > R || r < L) return;
    else if (L ≤ l && r ≤ R) {
        sum[i] = (sum[i] + (r - l + 1) * k) % p;
        add[i] = (add[i] + k) % p;
    }
    else {
        pushdown(i, l, r);
        Add(ls, l, mid, L, R, k);
        Add(rs, mid + 1, r, L, R, k);
        sum[i] = (sum[ls] + sum[rs]) % p;
    }
}
```




4

树状数组基础

Foundation of Binary Indexed Tree

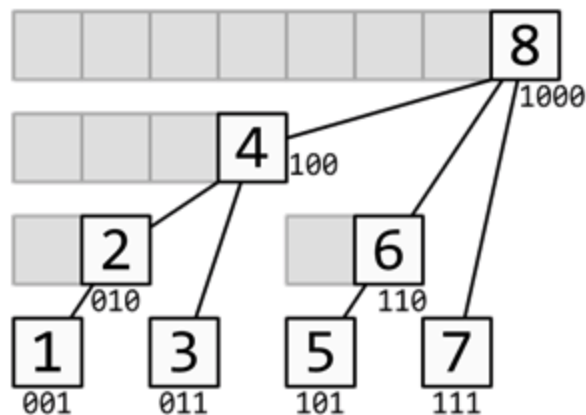
lowbit(i)

- lowbit(i) 函数即为 i 的二进制表示下的最低位1和后面的0构成的数值。
- 例如：
 - lowbit(9) = lowbit(1001₂) = 0001₂ = 1
 - lowbit(6) = lowbit(0010₂) = 0010₂ = 2
 - lowbit(24) = lowbit(11000₂) = 01000₂ = 8
- 根据位运算的规则，当我们对一个数取反(~)时，其 2 进制的 01 将被翻转。原数末尾的 0 经过翻转全部变为 1，其最低位的 1 经过翻转变成了 0。
- 此时若在取反的数上 +1，那么末尾的 1 将全部发生进位变成 0，第一个 0 会因为之前的进位变成 1。其余都保持不变。
 - 原数： 101011000
 - 取反： 010100111
 - 取反+1： 010101000
- 这时我们发现，原数与取反+1之后的数进行按位与操作，即可实现lowbit函数的功能，即：lowbit(i) = i & ((~i) + 1)
- 在计算机中，补码也是对原码进行了取反+1的操作，所以：
 - ($\sim i$) + 1 = -i
- 最终，lowbit(i) = i & (-i)

树状数组的结构

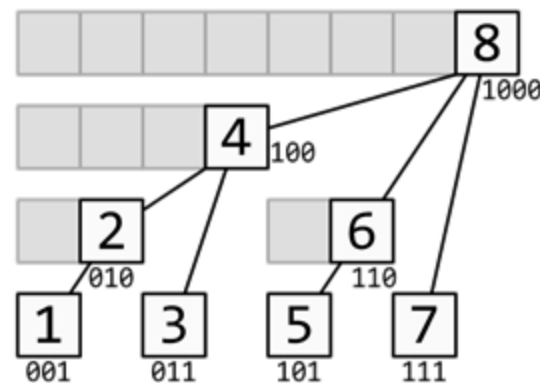
- 下面记 $lb_x = \text{lowbit}(x)$
- 树状数组是一棵多叉树，维护一个序列的信息，使用下标存储在数组中
- 下标为 i 的节点维护了序列 $(i - lb_x, i]$ 内的信息
- 以区间求和为例，设 $d[1\dots n]$ 为原序列，树状数组用来记录信息的数组为 $s[1\dots n]$ ，那么：

$$s[x] = \sum_{i=x-lb_x+1}^x d[i]$$



树状数组的结构特点

- 任何前缀 $[1, r]$ 都能被 $O(\log N)$ 个树状数组节点恰好覆盖
 - $s[r]$ 覆盖 $(r - 1b_r, r]$, 记 $x=r-1b_r$, 则 $s[x]$ 覆盖 $(x - 1b_x, x] \dots$
 - 每次迭代去掉一个二进制位, 最终只会用到 $O(\log N)$ 个节点
- 节点 x 的父节点编号是 $x + 1b_x$
 - 证明较长, 详见 oi-wiki 中树状数组的三个性质, 大概分别是说:
 - 1. x 的祖先节点编号一定大于 x
 - 2. $x + 1b_x$ 一定是 x 的祖先节点
 - 3. $(x, x + 1b_x)$ 内没有 x 的其他祖先节点了
 - 三点结合起来, 说明 $x + 1b_x$ 是 x 的父节点
- 节点 x 的父节点的 lowbit 严格大于 x 的 lowbit
 - 父节点编号为 $x + 1b_x$, 最低位 1 被进位了
 - 也因此, 树状数组树高不超过 $O(\log N)$





5

树 状 数 组 实 现

Implementation of Binary Indexed Tree

单点加法 区间求和

- 单点加法，只需要对该点到根的 $O(\log N)$ 个节点进行修改即可
- 求区间 $[1, r]$ 的和转换为求前缀 $[1, r]$ 和 $[1, 1-1]$ 的和，再做差
- 求前缀和只要迭代枚举 $O(\log N)$ 个节点求和即可
- 实现非常简单，常数非常优秀
- 时间复杂度 $O(\log N)$

```
struct BIT {
    int n;
    long long s[N];
    void add(int p, long long k) {
        for (; p ≤ n; p += p & -p) s[p] += k;
    }
    long long query(int p) {
        long long res = 0;
        for (; p; p -= p & -p) res += s[p];
        return res;
    }
    long long query(int l, int r) {
        return query(r) - query(l - 1);
    }
} bit;
```

```
int main() {
    int n, m;
    cin >> n >> m;
    bit.n = n;
    for (int i = 1, a; i ≤ n; ++i)
        cin >> a, bit.add(i, a);
    while (m--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1) bit.add(x, y);
        else cout << bit.query(x, y) << '\n';
    }

    return 0;
}
```

区间加法 单点求和

- 从维护原数组变为维护原数组的**差分数组**
- 由此，区间加法->两次单点加法，单点求和->前缀求和
- 以及还有用两个树状数组来维护区间加法、区间求和的技术

```
struct BIT {
    int n;
    long long s[N];
    void add(int p, long long k) {
        for (; p ≤ n; p += p & -p) s[p] += k;
    }
    long long query(int p) {
        long long res = 0;
        for (; p; p -= p & -p) res += s[p];
        return res;
    }
    void add(int l, int r, long long k) {
        add(l, k), add(r + 1, -k);
    }
} bit;
```

```
int main() {
    int n, m;
    cin >> n >> m;
    bit.n = n;
    for (int i = 1, a, las = 0; i ≤ n; ++i)
        cin >> a, bit.add(i, a - las), las = a;
    while (m--) {
        int op, x, y, k;
        cin >> op >> x;
        if (op == 1) cin >> y >> k, bit.add(x, y, k);
        else cout << bit.query(x) << '\n';
    }

    return 0;
}
```

区间加法 单点求和

- 从维护原数组变为维护原数组的**差分数组**
- 由此，区间加法->两次单点加法，单点求和->前缀求和
- 以及还有用两个树状数组来维护区间加法、区间求和的技术

```
struct BIT {
    int n;
    long long s[N];
    void add(int p, long long k) {
        for (; p ≤ n; p += p & -p) s[p] += k;
    }
    long long query(int p) {
        long long res = 0;
        for (; p; p -= p & -p) res += s[p];
        return res;
    }
    void add(int l, int r, long long k) {
        add(l, k), add(r + 1, -k);
    }
} bit;
```

```
int main() {
    int n, m;
    cin >> n >> m;
    bit.n = n;
    for (int i = 1, a, las = 0; i ≤ n; ++i)
        cin >> a, bit.add(i, a - las), las = a;
    while (m--) {
        int op, x, y, k;
        cin >> op >> x;
        if (op == 1) cin >> y >> k, bit.add(x, y, k);
        else cout << bit.query(x) << '\n';
    }

    return 0;
}
```

二维偏序

- 给定一个序列 $a_1, a_2, a_3, \dots, a_n$, 如果存在 $i < j$ 且 $a_i > a_j$, 那么我们称之为逆序对, 求给定序列中**逆序对的数目**。其中 $1 \leq a_i, n \leq 10^5$ 。
- 样例：输入：[4 1 3 2 5] 输出：4 解释：(4,1) (4,3),(4,2) (3,2)
- 对于任意一个 a_i , 统计在其之前且大于 a_i 的数的数量, 即为以 a_i 结尾的逆序对的数量。只要枚举 $i \in [1, n]$ 分别计算, 再求和, 即为所求答案。
- 使用树状数组维护桶, 记录每个数字出现的数量。从 a_1 到 a_n 依次插入, 并在每个数字插入前, 查询大于当前数字的数量之和。值域过大时需离散化。

```
int main() {
    int n;
    cin >> n;
    bit.n = 1e5;
    long long ans = 0;
    for (int i = 0, a; i < n; ++i) {
        cin >> a;
        ans += bit.query(a + 1, 1e5);
        bit.add(a, 1);
    }

    return 0;
}
```

时间复杂度: $O(N \log N)$

总结

- 线段树
 - 结构特征
 - 存储方式
 - 单点加法、区间求和
 - 懒标记
 - 区间加法、区间求和
 - 区间乘/加、区间求和
- 树状数组
 - 结构特征
 - 存储方式
 - 单点加法、区间求和
 - 区间加法、单点求和（维护差分）
 - 二维偏序（维护桶）
- 一些理念
 - 二者都是用空间换时间的算法
 - 二者都将 $O(N) \sim O(1)$ 平摊成了 $O(\log N) \sim O(\log N)$
- 树状数组和线段树相比？
 - 优势：
 - 实现非常简单
 - 常数表现优秀
 - 劣势：
 - 基于前缀做差，要求信息可差分
 - 选用：
 - 维护单点加法区间求和/区间加法单点求和用树状数组
 - 其他情况基本考虑线段树就可以了



为天下储人才
为国家图富强

感谢收听

Thank You For Your Listening